

Informe

Aprendizaje Supervisado

A partir de datos etiquetados : le das al modelo pares de entrada-salida y que él solo encuentre la función que los relaciona.

- . La salida es un número continuo: precio de una casa, temperatura mañana, etc

Aprendizaje como ajuste de función

- Tenés ejemplos: casas con sus características y sus precios reales
- El modelo ajusta una función $f(X)$ tal que $f(X) \approx Y$ para todos los ejemplos
- No se programan reglas, se generaliza comportamiento
- Cuantos más ejemplos y más representativos, mejor se generaliza,
- Componentes basicos:
- **X**: las entradas (features) — metros cuadrados, barrio, habitaciones
- **Y**: la salida real que queremos predecir — precio de la casa
- **Modelo**: la función parametrizada que hace la predicción — $\hat{y} = f(X, \theta)$
- **Predicción**: aplicar el modelo a un X nuevo para obtener $\hat{y}$
- **Función de pérdida**: mide qué tan lejos está $\hat{y}$ de Y — dice si el modelo es bueno o malo

Error y MSE

El error de una predicción es simplemente $Y - \hat{y}$. El MSE (Mean Squared Error) promedia los errores al cuadrado sobre todos los ejemplos:

$$MSE = (1/n) \times \sum (Y_i - \hat{y}_i)^2$$

Se eleva al cuadrado para que errores positivos y negativos no se cancelen, y para penalizar más los errores grandes. El objetivo del entrenamiento es minimizar este número.

Regresión lineal y modelos más flexibles

- **Regresión lineal:** asume que Y es una combinación lineal de las features. Simple, interpretable, rápida. Puede ser insuficiente si la relación real es curva.
- **Regresión polinómica:** agrega potencias de X (X^2 , X^3 ...) para capturar curvas. Más flexible pero más propensa a overfitting.
- **Otros modelos:** árboles de regresión, redes neuronales — hipótesis más complejas sobre cómo se relacionan X e Y .
- Elegir el modelo es elegir qué forma de relación asumís entre variables.

Ecuaciones normales: solución matemática exacta para regresión lineal. Se deriva la función de pérdida, se iguala a cero y se despeja θ . Exacta pero cara computacionalmente con muchos datos.

Gradiente descendente: método iterativo que funciona para cualquier modelo.

En cada paso:

1. Calculás el error actual con los parámetros actuales
2. Calculás el gradiente (la dirección en que crece el error)
3. Movés los parámetros en la dirección contraria
4. Repetís hasta converger

El tamaño del paso se llama **learning rate** — muy grande diverge, muy pequeño tarda mucho.

Generalización vs. memorización

- **Underfitting:** el modelo es demasiado simple, no captura el patrón. Error alto en entrenamiento y en datos nuevos.
- **Buen ajuste:** el modelo aprendió el patrón real. Error bajo en entrenamiento y en datos nuevos.
- **Overfitting:** el modelo memorizó los datos de entrenamiento incluyendo el ruido. Error muy bajo en entrenamiento, error alto en datos nuevos. El peor caso porque el modelo parece bueno pero no sirve.

Evaluación y diagnóstico:

Train, Validation y Test son las tres partes en que dividís los datos: entrenás en train, usás validation para tomar decisiones de diseño, y tocás test una sola vez

al final para no contaminar la evaluación. Cuando los datos son pocos, K-fold es más robusto porque rota cuál parte actúa como validation y aprovecha mejor cada ejemplo. Para diagnosticar el modelo usás curvas de aprendizaje: si el error en train y validation convergen alto hay underfitting, si divergen hay overfitting. Esa tensión es el bias-variance tradeoff: bias alto significa modelo demasiado simple, varianza alta significa que memorizó el entrenamiento y no generaliza. Para controlarlo se usa regularización, que penaliza parámetros grandes: L1 elimina features directamente, L2 los achica suavemente, ambas reducen overfitting.